

DATA ANALYTICS

07

Working with NumPy (Python)

Master the backbone of numerical computing — and make Python fast enough for real analytics.

DURATION	LEVEL	MODULE	UPDATED
9 Sessions	Intermediate	Module 7	June 2026

IN THIS MODULE

01	Introduction to NumPy & Why Analysts Use It
02	Creating Arrays
03	Array Attributes & Properties
04	Array Operations
05	Mathematical & Statistical Functions
06	Indexing & Slicing
07	Array Manipulation
08	Useful Utility Functions
09	Mini NumPy Case Study

SESSION

01

60 MIN · FUNDAMENTALS

Introduction to NumPy & Why Analysts Use It

◆ Learning Objectives

- Explain what NumPy is and why it is the foundation of Python-based data analytics
- Distinguish between a Python list and a NumPy array in terms of speed and memory
- Describe the concept of vectorisation and why it eliminates explicit loops
- Install and import NumPy correctly in a Jupyter Notebook
- Run a side-by-side benchmark comparing list vs array performance

01 What is NumPy and Where Does It Fit?

NumPy (Numerical Python) is the core library for numerical computing in Python. Every major analytics library — Pandas, Matplotlib, Seaborn, Scikit-learn — is built on top of NumPy arrays underneath. Think of it as the engine inside the car: you may not see it directly, but without it nothing moves.

In a typical analytics project — say, processing daily transaction records of a Surat textile exporter — you might have 500,000 rows of sales values. NumPy lets you compute the total, mean, or deviation across all rows in a single line, without writing a single for loop.

CONCEPT — NumPy ndarray

The core object in NumPy is the ndarray (n-dimensional array). Unlike a Python list, every element in an ndarray must be the same data type. This uniformity allows NumPy to store data as a compact block of memory — just like a C array — making operations dramatically faster.

02 Python List vs NumPy Array

Python List

Flexible — can hold mixed types (int, str, float). Stored as references to objects scattered in memory. Loops are required for element-wise math. Slow for large datasets.

NumPy Array

Homogeneous — all elements share one data type. Stored as a contiguous block in RAM. Supports vectorised operations — no loops needed. 10x–100x faster on large data.

```
import numpy as np
py_list = list(range(1_000_000))
np_arr = np.arange(1_000_000)
%timeit sum(py_list)
```

Import NumPy with the standard alias np
Create a Python list of 1 million integers
Create an equivalent NumPy array
Time the Python built-in sum (slow)

```
%timeit np_arr.sum()
```

Time NumPy's sum (fast — vectorised C code)

TRY THIS LIVE

Run the benchmark above in your Jupyter Notebook. Note the difference in microseconds. Then try `np_arr * 2` in one line — observe that every element doubles with no loop.

03 Vectorisation — The Core Idea

Vectorisation means applying an operation to an entire array at once instead of element by element. NumPy pushes this computation down to pre-compiled C and Fortran code, bypassing Python's interpreter overhead. For a data analyst working with lakh-row datasets, this is not a luxury — it is a necessity.

CONCEPT — Vectorisation

Instead of: `for i in range(len(prices)): gst[i] = prices[i] * 0.18`

You write: `gst = prices * 0.18`

Both produce the same result. The NumPy version runs orders of magnitude faster because it executes in compiled C, not Python.

CASE STUDY — Flipkart Big Billion Days Pricing Engine

Context — Flipkart, India's largest homegrown e-commerce platform, runs flash sales with millions of concurrent SKU price updates during Big Billion Days.

Challenge — Recomputing discounted prices, GST components, and margin flags for 2 million+ SKUs in near real-time was too slow with row-by-row Python loops.

What they did — Their data engineering team used NumPy vectorised operations to compute price transformations — applying discount percentages, rounding rules, and margin thresholds across entire arrays in a single pass.

Result — Batch price-update jobs that previously took minutes ran in seconds, allowing pricing models to refresh every few minutes during live sales.

Takeaway — Vectorisation is not just a performance trick — at scale it is the difference between a system that works and one that cannot keep up.

Source: Flipkart Engineering Blog — Data Infrastructure at Scale (2022)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Week 1: Introduction (IIT Madras,

Prof. Madhavan Mukund)

Link — <https://nptel.ac.in/courses/106106145>

Always link the official channel — beware fan-made or re-upload copies.

04 Installing & Importing NumPy

```
pip install numpy
import numpy as np
print(np.__version__)
```

Install NumPy (already included in Anaconda)

Standard import — always use alias np

Verify your installed version (expect 1.24+ or 2.x)

SPEED ADVANTAGE OVER PYTHON LISTS

~100×

Source: NumPy Official Benchmarks — numpy.org/doc

“NumPy is to data analytics what a calculator is to an accountant — not optional, just essential.”

WHERE TO GO NEXT

You now understand why NumPy exists. Next, you'll learn how to actually create arrays of every shape and size.

Explore deeper — NumPy official quickstart: <https://numpy.org/doc/stable/user/quickstart.html>

Try on your own — Create a NumPy array of 10 random employee salaries and compute their sum, mean, and max in a single notebook cell each.

Connect the dots — Session 02 covers all the ways to create arrays — from manual input to zeros, ones, random, and range-based arrays.

Going further — Read about NumPy's memory layout (C-contiguous vs Fortran-contiguous) and how it affects cache performance.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor for a data analytics student. Explain the difference between a Python list and a NumPy array in simple Hinglish. Use an example of calculating GST on 1 lakh product prices. Keep it under 150

words.

SESSION

02

60 MIN · FUNDAMENTALS

Creating Arrays

◆ Learning Objectives

- Create arrays from Python lists using `np.array()`
- Generate range-based arrays with `np.arange()` and `np.linspace()`
- Build constant arrays using `np.zeros()`, `np.ones()`, and `np.empty()`
- Construct identity and diagonal matrices with `np.eye()` and `np.identity()`
- Generate random arrays using `np.random` functions for simulation and testing

01 Creating Arrays from Existing Data

The most direct way to create a NumPy array is to convert an existing Python list using `np.array()`. You can create 1D arrays (a single row of values), 2D arrays (rows and columns — like a spreadsheet), or higher-dimensional arrays depending on the nested structure of the list you pass in.

```
import numpy as np                                Standard import
sales = np.array([15000, 22000, 18500, 31000, 27000])
    1D array — 5 daily sales figures
matrix = np.array([[1, 2, 3], [4, 5, 6]])          2D array — 2 rows, 3 columns
print(sales.dtype)                                Check inferred data type — likely int64
float_arr = np.array([1.5, 2.3, 3.7], dtype=np.float32)
    Force a specific data type
```

02 Range & Sequence Arrays

CONCEPT — `arange` vs `linspace`

`np.arange(start, stop, step)` works like Python's `range()` but returns an array — you control the step size.

`np.linspace(start, stop, num)` is different: you specify how many evenly-spaced points you want between start and stop. Use `arange` when you know the step; use `linspace` when you know the count.

<code>np.arange(0, 100, 10)</code>	Array: 0, 10, 20, ... 90 (step of 10)
<code>np.arange(1, 13)</code>	Months 1 to 12 — for a yearly sales index
<code>np.linspace(0, 1, 5)</code>	5 points evenly spaced from 0 to 1: [0, 0.25, 0.5, 0.75, 1.0]
<code>np.linspace(100, 500, 9)</code>	9 price points evenly spaced from Rs 100 to Rs 500
<code>np.logspace(1, 4, 4)</code>	Logarithmic scale: [10, 100, 1000, 10000] — useful for exponential growth

TRY THIS LIVE

Create a linspace array of 12 evenly-spaced monthly sales targets between Rs 50,000 and Rs 2,00,000 and print it. Then use arange to generate quarter labels [1, 2, 3, 4].

03 Constant & Structural Arrays

Often you need a pre-filled array as a starting point — a grid of zeros to accumulate results into, or an identity matrix for linear algebra. NumPy provides dedicated functions for each of these common patterns.

<code>np.zeros((3, 4))</code>	3×4 matrix of all zeros — useful as an accumulator
<code>np.ones((2, 5))</code>	2×5 matrix of all ones
<code>np.full((3, 3), 7)</code>	3×3 matrix filled with the value 7
<code>np.empty((2, 2))</code>	Uninitialised 2×2 array (garbage values) — fastest to create, use carefully
<code>np.eye(4)</code>	4×4 identity matrix (1s on diagonal, 0s elsewhere)
<code>np.identity(3)</code>	Same as <code>eye(3)</code> — square identity matrix

04 Random Arrays

Generating random arrays is essential for simulation, testing, and bootstrapping. NumPy's random module offers multiple distributions — uniform, normal (Gaussian), and integer random sampling — all of which appear constantly in analytics work.

<code>np.random.seed(42)</code>	Fix the seed for reproducibility — always set in notebooks
<code>np.random.rand(5)</code>	5 random floats uniformly distributed between 0 and 1
<code>np.random.randn(3, 3)</code>	3×3 matrix of values from a standard normal distribution (mean=0, std=1)
<code>np.random.randint(1, 101, size=10)</code>	10 random integers between 1 and 100 — simulate survey scores
<code>np.random.choice([10, 20, 30, 40], size=6, replace=True)</code>	Random sample from a list — simulate dice roll or product selection

PRO TIP — Always Set a Random Seed

`np.random.seed(42)` at the top of your notebook ensures that anyone who runs your code gets the same random numbers. This makes analysis reproducible — a non-negotiable requirement in professional analytics reporting.

Function	What it creates	Typical analyst use
<code>np.array()</code>	Array from list/tuple	Convert loaded data
<code>np.arange()</code>	Range with fixed step	Index arrays, time steps
<code>np.linspace()</code>	Fixed-count even spacing	Price grids, model inputs
<code>np.zeros()</code> / <code>np.ones()</code>	Constant-filled array	Initialise output arrays
<code>np.random.randn()</code>	Normal distribution	Simulate sales noise
<code>np.random.randint()</code>	Random integers	Synthetic test data

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — NumPy section (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now build arrays of any shape. Next you'll learn to read their properties — shape, size, dtype — and reshape them.

Explore deeper — NumPy array creation routines:

<https://numpy.org/doc/stable/reference/routines.array-creation.html>

Try on your own — Create a 5×3 random integer array (values 50–500) simulating weekly sales for 5 products across 3 cities in Gujarat.

Connect the dots — Session 03 covers array attributes — how to inspect and reshape what you've created.

Going further — Explore `np.random.Generator` (the modern NumPy random API, preferred over legacy `np.random.*` since NumPy 1.17).

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a NumPy mentor. In simple Hinglish, explain the difference between `np.arange()` and `np.linspace()` using an example of creating monthly sales targets for a Surat garment company. Keep it under 150 words.

SESSION

03

60 MIN · FUNDAMENTALS

Array Attributes & Properties

◆ Learning Objectives

- Read key array attributes: `ndim`, `shape`, `size`, `dtype`, `itemsize`, and `nbytes`
- Reshape arrays using `reshape()`, `ravel()`, `flatten()`, and `resize()`
- Transpose arrays using `.T` and `np.transpose()`
- Explain the difference between a view and a copy when reshaping
- Apply reshaping to real analytical scenarios such as restructuring tabular data

01 Reading Array Attributes

Before you operate on a NumPy array, you should always inspect its properties. Knowing the shape tells you how many rows and columns exist; knowing the `dtype` tells you what kind of numbers are stored and how much memory each element uses. These checks prevent silent bugs — especially when data arrives from CSV files or APIs.

<code>import numpy as np</code>	Standard import
<code>arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90], [100, 110, 120]])</code>	
4-row, 3-column array — imagine 4 regions × 3 months of sales	
<code>arr.ndim</code>	Number of dimensions -> 2
<code>arr.shape</code>	Tuple of (rows, columns) -> (4, 3)
<code>arr.size</code>	Total number of elements -> 12
<code>arr.dtype</code>	Data type of elements -> int64 (on most systems)
<code>arr.itemsize</code>	Bytes per element -> 8 (for int64)
<code>arr.nbytes</code>	Total memory used -> size × itemsize = 96 bytes

CONCEPT — dtype matters

float64 uses 8 bytes per element; float32 uses 4. For a dataset with 10 million rows, switching from float64 to float32 halves your memory footprint from ~80 MB to ~40 MB. This matters on cloud VMs with limited RAM — a common situation in TOPS alumni's first analytics jobs.

02 Reshaping Arrays

`reshape()` returns a new view of the same data with a different shape — no copying occurs. The total number of elements must remain the same. This is commonly used to feed data into machine learning models that expect specific input shapes.

<code>arr_1d = np.arange(1, 13)</code>	Flat array of 12 values: [1, 2, ... 12]
<code>arr_2d = arr_1d.reshape(4, 3)</code>	Reshape to 4 rows × 3 columns — still 12 elements
<code>arr_2d.reshape(2, 6)</code>	Reshape again to 2 rows × 6 columns
<code>arr_1d.reshape(3, -1)</code>	Use -1 to let NumPy auto-calculate the missing dimension -> (3, 4)
<code>arr_2d.ravel()</code>	Flatten to 1D — returns a VIEW (fast, shares memory)
<code>arr_2d.flatten()</code>	Flatten to 1D — returns a COPY (safe, independent)
<code>np.resize(arr_1d, (3, 5))</code>	Resize and repeat values if new size is larger — returns a copy

PRO TIP — View vs Copy

`ravel()` returns a view — modifying the result modifies the original array. `flatten()` always returns a copy — modifying it leaves the original untouched. In production code, use `flatten()` when you want safety and `ravel()` only when you need performance and are certain you won't modify the result.

TRY THIS LIVE

Create `np.arange(1, 25)` and reshape it to (4, 6). Then print its shape, size, and nbytes. Finally flatten it and confirm the shape goes back to (24,).

03 Transposing Arrays

Transposing swaps rows and columns — turning a (4, 3) array into a (3, 4) array. This is essential when aligning matrices for multiplication or reshaping data before feeding it into a model. NumPy transpose is always a view, not a copy.

<code>sales = np.array([[100, 200, 300], [400, 500, 600]])</code>	
2 rows (products) × 3 columns (months)	
<code>sales.T</code>	Transpose -> 3 rows (months) × 2 columns (products)
<code>np.transpose(sales)</code>	Equivalent to <code>sales.T</code> — same result
<code>sales.T.shape</code>	-> (3, 2)

Operation	Returns	Modifies original?
<code>reshape()</code>	View	Yes (shared memory)
<code>ravel()</code>	View (usually)	Yes (shared memory)
<code>flatten()</code>	Copy	No
<code>resize()</code>	Copy	No

Operation	Returns	Modifies original?
.T / transpose()	View	Yes (shared memory)

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Array operations (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now inspect and reshape any array. Next you will perform arithmetic, comparison, and logical operations directly on arrays.

Explore deeper — NumPy shape manipulation docs:

<https://numpy.org/doc/stable/reference/routines.array-manipulation.html>

Try on your own — Create a (12, 4) array of random integers (simulate 12 months × 4 branches of sales). Transpose it, then flatten it and check nbytes.

Connect the dots — Session 04 covers operations — adding, subtracting, multiplying, comparing arrays element-wise and with broadcasting.

Going further — Read about np.newaxis and how it is used to add dimensions for broadcasting compatibility.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a NumPy coach. In simple Hinglish, explain the difference between reshape(), ravel(), and flatten() with a real example of restructuring quarterly sales data for 4 zones. Keep it under 150 words.

SESSION

04

65 MIN · CORE SKILLS

Array Operations

◆ Learning Objectives

- Apply arithmetic operators (+, -, *, /, //, **, %) element-wise on arrays
- Use comparison operators to produce boolean arrays
- Apply logical operations with np.logical_and(), np.logical_or(), np.logical_not()
- Explain broadcasting and apply it to operate on arrays of different shapes
- Distinguish between element-wise and aggregate operations and know when to use each

01 Arithmetic Operations

Every standard arithmetic operator works element-wise on NumPy arrays — no loop required. When you add two arrays of the same shape, corresponding elements are added. This makes operations like adding a tax rate across all prices, or computing profit margins across all rows, a single concise expression.

<code>import numpy as np</code>	Standard import
<code>prices = np.array([500, 1200, 850, 3000, 450])</code>	
Product prices in Rs	
<code>qty = np.array([10, 5, 20, 2, 15])</code>	Quantities sold
<code>revenue = prices * qty</code>	Element-wise multiply -> [5000, 6000, 17000, 6000, 6750]
<code>gst = prices * 0.18</code>	Broadcast scalar: add 18% GST to every price
<code>discount = prices - (prices * 0.10)</code>	Apply 10% discount to all prices
<code>prices ** 2</code>	Square every price
<code>revenue // 1000</code>	Integer division — revenue in thousands
<code>revenue % 1000</code>	Remainder — amount below nearest thousand

TRY THIS LIVE

Create an array of 6 employee CTC values (in lakhs). Calculate take-home after 30% tax deduction in a single line. Then compute the bonus as 10% of CTC and add it back — all without a single loop.

02 Comparison Operators

Comparison operators applied to a NumPy array return a boolean array of the same shape — True where the condition holds, False elsewhere. These boolean arrays are the backbone of filtering operations in both NumPy and Pandas.

```
scores = np.array([72, 85, 45, 91, 60, 38, 78])
```

Student exam scores

```
scores >= 75
```

-> [F, T, F, T, F, F, T] — who passed with distinction

```
scores == 45
```

-> exact match — returns bool array

```
scores != 60
```

-> all scores except 60

```
(scores >= 60) & (scores < 80)
```

AND — scores between 60 and 79

```
scores[scores >= 75]
```

Boolean indexing — filter to only distinction scores

03 Logical Operations

```
active = np.array([True, True, False, True, False])
```

Is customer active?

```
premium = np.array([False, True, False, True, True])
```

Is customer premium?

```
np.logical_and(active, premium)
```

Active AND premium -> [F, T, F, T, F]

```
np.logical_or(active, premium)
```

Active OR premium -> [T, T, F, T, T]

```
np.logical_not(active)
```

NOT active -> [F, F, T, F, T]

04 Broadcasting

CONCEPT — Broadcasting

Broadcasting allows NumPy to operate on arrays of different shapes by 'stretching' the smaller array across the larger one — without actually copying data. Rule of thumb: two dimensions are compatible if they are equal, or one of them is 1. A scalar (single number) can always broadcast against any array.

```
sales = np.array([[100, 200, 300], [400, 500, 600]])
```

Shape (2, 3) — 2 regions × 3 months

```
targets = np.array([150, 250, 350])
```

Shape (3,) — one target per month

```
sales - targets
```

Broadcasts targets across both rows -> variance per region per month

```
sales * 1.10
```

Scalar broadcast — apply 10% growth to all values

Element-wise Operation

Operates on each element independently. Result has the same shape as the input. Examples: `arr * 2`, `arr + arr`, `np.sqrt(arr)`. Used when you need per-element results.

Aggregate Operation

Collapses one or more dimensions to produce a summary. Result is smaller than input. Examples: `arr.sum()`, `arr.mean()`, `arr.max()`. Used when you need row/column/total summaries.

<code>sales.sum()</code>	Total of all elements
<code>sales.sum(axis=0)</code>	Column-wise sum (sum each month across regions)
<code>sales.sum(axis=1)</code>	Row-wise sum (total per region across months)
<code>sales.mean(axis=0)</code>	Average monthly performance across regions

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Numerical operations (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now perform any arithmetic, comparison, or logical operation on arrays. Next session expands this to a full library of mathematical and statistical functions.

Explore deeper — NumPy broadcasting guide: <https://numpy.org/doc/stable/user/basics.broadcasting.html>

Try on your own — Build a 4×3 array of daily revenue for 4 products across 3 days. Apply a 5% weekend surcharge to the last column only using broadcasting.

Connect the dots — Session 05 covers statistical functions — mean, median, std, percentile, and more — that aggregate arrays into insights.

Going further — Study `np.einsum()` — an advanced function for generalised array operations used in deep learning frameworks.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a data analytics trainer. Explain NumPy broadcasting in simple Hinglish using the example of applying different GST rates (5%, 12%, 18%) to a list of product prices in a single operation. Keep it under 150 words.

SESSION

05

65 MIN · CORE SKILLS

Mathematical & Statistical Functions

◆ Learning Objectives

- Apply core math functions: `np.sqrt()`, `np.exp()`, `np.log()`, `np.power()`, `np.abs()`
- Use rounding functions: `np.round()`, `np.floor()`, `np.ceil()`
- Compute descriptive statistics: mean, median, std, var, min, max, sum, cumsum, percentile
- Apply trigonometric functions and understand their use in signal/cyclical data
- Use axis parameter to compute row-wise and column-wise statistics

01 Core Mathematical Functions

NumPy provides universal functions (ufuncs) — vectorised wrappers around C-level math operations. They apply element-wise to any array and are far faster than calling Python's math module in a loop. For analytics work, functions like `np.sqrt()` and `np.log()` appear constantly in feature engineering and data transformation.

<code>import numpy as np</code>	Standard import
<code>values = np.array([4, 9, 16, 25, 100])</code>	Sample array
<code>np.sqrt(values)</code>	Square root of each element -> [2, 3, 4, 5, 10]
<code>np.power(values, 2)</code>	Square each element -> [16, 81, 256, 625, 10000]
<code>np.abs(np.array([-5, 3, -1, 8]))</code>	Absolute values -> [5, 3, 1, 8]
<code>np.exp(np.array([0, 1, 2]))</code>	Exponential e^x -> [1, 2.718, 7.389]
<code>np.log(np.array([1, np.e, 100]))</code>	Natural log -> [0, 1, 4.605]
<code>np.log10(np.array([1, 10, 100, 1000]))</code>	Log base 10 -> [0, 1, 2, 3]

CONCEPT — Log Transformation

In analytics, skewed data (like income or house prices) is often log-transformed before modelling. `np.log(prices)` compresses large values and makes distribution more normal. This is one of the most common pre-processing steps in regression and ML feature engineering.

02 Rounding Functions

<code>prices = np.array([99.456, 249.999, 1500.123, 49.501])</code>	
Raw float prices	
<code>np.round(prices, 2)</code>	Round to 2 decimal places -> [99.46, 250.0, 1500.12, 49.5]

```
np.floor(prices)
np.ceil(prices)
```

Round DOWN to nearest integer -> [99, 249, 1500, 49]

Round UP to nearest integer -> [100, 250, 1501, 50]

03 Statistical Functions

These are the functions you will use most often as a data analyst. They summarise arrays into meaningful single numbers — or, when you specify an axis, into per-row or per-column summaries. Mastering the axis parameter is crucial: axis=0 operates column-wise (collapses rows); axis=1 operates row-wise (collapses columns).

```
sales = np.array([[1200, 1500, 1100], [900, 1800, 1300], [2000, 1600, 1400]])
```

3 branches × 3 months sales

```
np.mean(sales)
```

Overall average -> scalar

```
np.mean(sales, axis=0)
```

Average per month (across branches)

```
np.mean(sales, axis=1)
```

Average per branch (across months)

```
np.median(sales)
```

Median of all values

```
np.std(sales)
```

Standard deviation — spread around mean

```
np.var(sales)
```

Variance — squared std

```
np.min(sales)
```

Minimum value in entire array

```
np.max(sales, axis=1)
```

Best month per branch

```
np.sum(sales, axis=0)
```

Total revenue per month

```
np.cumsum(sales[0])
```

Cumulative sum for branch 0 across months

```
np.prod(np.array([1.05, 1.08, 1.10]))
```

Compound growth factor: 5%, 8%, 10%

```
np.percentile(sales, 75)
```

75th percentile — top-quartile sales threshold

FUNCTIONS IN NUMPY.UFUNC LIBRARY

60+

Source: NumPy Official Docs — numpy.org/doc/stable/reference/ufuncs.html

CASE STUDY — IFFCO Fertiliser Pricing Analytics

Context — IFFCO (Indian Farmers Fertiliser Cooperative), one of India's largest fertiliser producers, sells across thousands of regional distribution points.

Challenge — The analytics team needed to compute regional price variance, identify outlier districts with abnormal pricing, and forecast compound price growth for the next 3 years.

What they did — Using NumPy statistical functions — np.std(), np.percentile(), and np.prod() for compound growth — the team processed district-level pricing arrays and flagged values beyond the 90th percentile as pricing anomalies requiring audit.

Result — The pricing audit identified 12% of distribution points with irregularities, leading to policy

corrections that saved Rs 4 crore in procurement cost variances.

Takeaway — `np.percentile()` and `np.std()` on arrays of price data can surface anomalies faster than any manual review process.

Source: IFFCO Annual Report 2022-23; analytics methodology reconstructed for educational purposes

TRY THIS LIVE

Create a 5×4 random integer array (values 10,000 to 50,000) representing monthly revenue for 5 products × 4 quarters. Compute mean per product (`axis=1`), std per quarter (`axis=0`), and identify which quarter had the highest total revenue.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Statistical operations (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now compute any mathematical or statistical summary on arrays. Next you will learn to access and filter specific elements using indexing and slicing.

Explore deeper — NumPy statistics reference:
<https://numpy.org/doc/stable/reference/routines.statistics.html>

Try on your own — Take a real dataset column from a CSV (e.g. salary column) — load it with `np.genfromtxt()` and compute min, max, mean, and 90th percentile.

Connect the dots — Session 06 covers indexing and slicing — how to extract exactly the rows and columns you need from any array.

Going further — Study `np.histogram()` for building frequency distributions, and `np.corrcoef()` for correlation matrices — both are used in EDA.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a statistics trainer. In simple Hinglish, explain the difference between `np.mean()` and `np.median()` with an example of monthly salary data for employees at an Ahmedabad IT company where a few senior salaries are very high. Keep it under 150 words.

SESSION

06

65 MIN · CORE SKILLS

Indexing & Slicing

Learning Objectives

- Access individual elements using basic integer indexing in 1D and 2D arrays
- Slice arrays using the start:stop:step syntax to extract subsets
- Use negative indexing to access elements from the end of an array
- Apply fancy indexing to select multiple non-contiguous elements at once
- Use boolean indexing to filter arrays based on conditions
- Apply np.where() to conditionally select or replace values

01 Basic Indexing

Indexing in NumPy is zero-based — the first element is at index 0. For 2D arrays, you specify the row index first, then the column index. You can think of a 2D array as a table: `arr[row, col]` is the value at that cell. This is identical in concept to selecting a cell in Excel by row and column number.

<code>import numpy as np</code>	Standard import
<code>arr = np.array([10, 20, 30, 40, 50])</code>	1D array of 5 values
<code>arr[0]</code>	First element -> 10
<code>arr[4]</code>	Last element -> 50
<code>arr[-1]</code>	Last element using negative index -> 50
<code>arr[-2]</code>	Second to last -> 40
<code>matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])</code>	3x3 matrix
<code>matrix[1, 2]</code>	Row 1, Column 2 -> 6
<code>matrix[0, 0]</code>	Top-left corner -> 1
<code>matrix[-1, -1]</code>	Bottom-right corner -> 9

02 Slicing

CONCEPT — Slice Syntax

`arr[start:stop:step]` — start is inclusive, stop is exclusive, step is the jump. Leaving any part blank means: start=beginning, stop=end, step=1. This is identical to Python list slicing but works on multiple dimensions simultaneously in NumPy.

<code>arr = np.arange(10, 110, 10)</code>	-> [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
<code>arr[2:6]</code>	Elements at index 2,3,4,5 -> [30, 40, 50, 60]

<code>arr[:4]</code>	First 4 elements -> [10, 20, 30, 40]
<code>arr[6:]</code>	From index 6 to end -> [70, 80, 90, 100]
<code>arr[::2]</code>	Every other element -> [10, 30, 50, 70, 90]
<code>arr[::-1]</code>	Reverse the entire array
<code>matrix[0:2, 1:3]</code>	Rows 0-1, Columns 1-2 — a 2x2 sub-matrix
<code>matrix[:, 1]</code>	All rows, column 1 only — extract a single column
<code>matrix[1, :]</code>	Row 1, all columns — extract a single row

TRY THIS LIVE

Create a 5x5 array using `np.arange(1, 26).reshape(5, 5)`. Extract the top-right 2x2 corner. Then extract the middle row. Then extract every other column.

03 Fancy Indexing

Fancy indexing lets you select multiple non-contiguous elements by passing a list or array of indices. Unlike slicing (which always returns a view), fancy indexing always returns a copy of the data.

<code>products = np.array([100, 200, 300, 400, 500, 600])</code>	
6 product prices	
<code>products[[0, 2, 4]]</code>	Select items at index 0, 2, 4 -> [100, 300, 500]
<code>products[[3, 3, 1]]</code>	Repeat indices allowed -> [400, 400, 200]
<code>matrix[[0, 2], :]</code>	Select rows 0 and 2 from a 2D array

04 Boolean Indexing & `np.where()`

Boolean indexing is one of the most powerful features in NumPy — and the same pattern carries directly into Pandas. You create a boolean condition on an array, and then use that boolean array as the index to filter only the matching elements.

<code>salaries = np.array([25000, 45000, 72000, 38000, 91000, 55000])</code>	
Monthly salaries in Rs	
<code>salaries[salaries > 50000]</code>	Filter: only salaries above Rs 50,000 -> [72000, 91000, 55000]
<code>salaries[(salaries >= 30000) & (salaries <= 60000)]</code>	
Range filter -> [45000, 38000, 55000]	
<code>np.where(salaries > 50000, 'Senior', 'Junior')</code>	
Conditional label — <code>np.where(condition, if_true, if_false)</code>	

```
np.where(salaries > 50000, salaries * 1.10, salaries)
```

Give 10% raise only to Senior staff

PRO TIP — np.where() in Feature Engineering

np.where() is the NumPy equivalent of Excel's IF() function — but vectorised. It is widely used to create new derived columns: flag transactions above threshold, classify customers by age group, bin income ranges. You will use this constantly once you move into Pandas and ML preprocessing.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Indexing and slicing (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now extract any subset of an array with precision. Next session covers manipulation — stacking, splitting, inserting, and deleting arrays.

Explore deeper — NumPy indexing guide: <https://numpy.org/doc/stable/user/basics.indexing.html>

Try on your own — Load a list of 20 random exam scores (np.random.randint(40, 100, 20)). Use boolean indexing to find all failing scores (<60). Use np.where() to label each as Pass or Fail.

Connect the dots — Session 07 covers array manipulation — combining multiple arrays, splitting them, and inserting/removing elements.

Going further — Study np.ix_() for open mesh indexing — useful for selecting from a 2D array using two independent index lists.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python data analytics mentor. In simple Hinglish, explain the difference between slicing and boolean indexing in NumPy using the example of filtering customer transactions above Rs 10,000 from a Jio Payments Bank dataset. Keep it under 150 words.

SESSION

07

65 MIN · CORE SKILLS

Array Manipulation

◆ Learning Objectives

- Combine arrays along axes using `np.concatenate()`, `np.vstack()`, and `np.hstack()`
- Stack arrays depth-wise with `np.dstack()`
- Split arrays into equal or unequal parts using `np.split()` and `np.array_split()`
- Insert and delete elements using `np.insert()`, `np.append()`, and `np.delete()`
- Distinguish between a view and a copy and use `.copy()` correctly

01 Combining Arrays

In real analytics work, data often arrives in separate arrays or DataFrames that need to be combined — monthly sales from different branches, or feature sets from different sources. NumPy provides several stacking and concatenation functions, each suited for a different arrangement.

```
import numpy as np          Standard import
a = np.array([1, 2, 3])     First array
b = np.array([4, 5, 6])     Second array
np.concatenate([a, b])      Join end-to-end -> [1,2,3,4,5,6]
np.concatenate([a.reshape(1, -1), b.reshape(1, -1)], axis=0)
    Concatenate as rows (axis=0)
np.concatenate([a.reshape(-1, 1), b.reshape(-1, 1)], axis=1)
    Concatenate as columns (axis=1)
```

CONCEPT — `vstack`, `hstack`, `dstack`

`vstack` (vertical stack) stacks arrays row-wise — adds new rows. `hstack` (horizontal stack) stacks column-wise — adds new columns. `dstack` (depth stack) stacks along a third axis — creates 3D arrays. For 2D analytical data, `vstack` = 'add more records'; `hstack` = 'add more features'.

```
jan = np.array([[100, 200], [300, 400]])    January sales: 2 products × 2 regions
feb = np.array([[150, 250], [350, 450]])    February sales: same structure
np.vstack([jan, feb])                       Stack rows -> 4×2 combined monthly records
np.hstack([jan, feb])                       Stack columns -> 2×4 (adds months as columns)
np.dstack([jan, feb])                       Stack depth -> 2×2×2 (3D: product × region × month)
```

TRY THIS LIVE

Create three 3x2 arrays representing Q1, Q2, Q3 sales for 3 products × 2 cities. Stack them vertically to build a full-year record array. Then extract Q2 back out by slicing.

02 Splitting Arrays

<code>arr = np.arange(1, 13)</code>	12-element array: [1, 2, ... 12]
<code>np.split(arr, 3)</code>	Split into 3 equal parts -> [[1-4], [5-8], [9-12]]
<code>np.split(arr, [3, 7])</code>	Split at index 3 and 7 -> [[1-3], [4-7], [8-12]]
<code>np.array_split(arr, 5)</code>	Split into 5 — unequal parts allowed (useful when size not divisible)
<code>matrix = np.arange(1, 13).reshape(4, 3)</code>	4x3 matrix
<code>np.vsplit(matrix, 2)</code>	Split vertically into 2 halves -> two 2x3 arrays
<code>np.hsplit(matrix, 3)</code>	Split horizontally into 3 parts -> three 4x1 arrays

03 Inserting, Appending & Deleting

<code>arr = np.array([10, 20, 30, 40, 50])</code>	Base array
<code>np.insert(arr, 2, 99)</code>	Insert 99 at index 2 -> [10, 20, 99, 30, 40, 50]
<code>np.append(arr, [60, 70])</code>	Append values to end -> [10, 20, 30, 40, 50, 60, 70]
<code>np.delete(arr, 1)</code>	Delete element at index 1 -> [10, 30, 40, 50]
<code>np.delete(arr, [0, 4])</code>	Delete first and last -> [20, 30, 40]

WARNING — `np.append()` Creates a New Array

`np.append()` does NOT modify the array in-place. It returns a brand new array every time. Calling it inside a loop is very inefficient (creates n copies). For accumulating large datasets, build a Python list first and convert with `np.array()` at the end.

04 View vs Copy

View (.view())

Points to the same memory block as the original. Modifying the view modifies the original. Fast — no new memory allocated. Created by: slicing, reshape(), ravel(), transpose().

Copy (.copy())

Allocates a completely independent block of memory. Modifying the copy never affects the original. Slightly slower. Created by: flatten(), fancy indexing, np.copy(), boolean indexing.

```
original = np.array([1, 2, 3, 4, 5])
view = original[1:4]
view[0] = 99
safe = original.copy()
safe[0] = 0
```

Source array
 Slice -> a VIEW (shares memory with original)
 Modify view -> original[1] also becomes 99!
 Create a true independent copy
 Modify copy -> original is untouched

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Array operations (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You can now combine, split, and safely modify arrays. Next session covers NumPy's utility functions for sorting, deduplication, NaN handling, and clipping.

Explore deeper — NumPy array manipulation docs:
<https://numpy.org/doc/stable/reference/routines.array-manipulation.html>

Try on your own — Simulate 3 months of daily sales (30 values each) as 3 separate arrays. Stack them into a single 90-day array. Then split it back into weeks (np.array_split into 13 parts).

Connect the dots — Session 08 covers utility functions — sort, unique, clip, NaN handling — that are essential for data cleaning.

Going further — Study np.block() — a powerful function for assembling arrays from nested lists of blocks, useful in matrix construction.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Python tutor. In simple Hinglish, explain the difference between a NumPy view and a copy using an example where an analyst accidentally modifies the original sales array while cleaning a sliced subset. Keep it under 150 words.

SESSION

08

60 MIN · DATA CLEANING

Useful Utility Functions

◆ Learning Objectives

- Extract unique values and their counts using `np.unique()`
- Sort arrays with `np.sort()` and retrieve sort order indices with `np.argsort()`
- Locate elements matching a condition using `np.where()` and `np.nonzero()`
- Test array-wide conditions with `np.any()` and `np.all()`
- Detect and handle NaN and infinite values using `np.isnan()`, `np.isinf()`, and nanfunctions
- Clip outlier values to a valid range using `np.clip()`

01 Unique Values & Sorting

Before analysing a dataset, you almost always need to understand what unique values exist in a column, and whether the data is sorted in a meaningful order. NumPy's `np.unique()` and `np.sort()` cover both needs and run at C speed even on million-row arrays.

```
import numpy as np                                Standard import
categories = np.array(['Mumbai', 'Surat', 'Mumbai', 'Ahmedabad', 'Surat', 'Mumbai'])
City column with repeats
np.unique(categories)                             -> ['Ahmedabad', 'Mumbai', 'Surat'] — sorted unique
                                                values
vals, counts = np.unique(categories, return_counts=True)
Also return frequency of each unique value
scores = np.array([72, 45, 91, 38, 85, 60])        Student scores
np.sort(scores)                                   Sorted ascending -> [38, 45, 60, 72, 85, 91]
np.sort(scores)[::-1]                             Sorted descending -> [91, 85, 72, 60, 45, 38]
np.argsort(scores)                                Returns indices that would sort the array -> [3, 1, 5, 0,
                                                4, 2]
names = np.array(['Rohan', 'Priya', 'Arjun', 'Meera', 'Siddhi', 'Kartik'])
Student names
names[np.argsort(scores)]                         Rank students by score using argsort -> sorted name
                                                list
```

CONCEPT — argsort for Ranking

`np.argsort()` returns the indices that would sort an array, not the sorted values themselves. This is powerful: pass those indices into another array (like names) to sort it by the scores without zipping or using a dictionary. This pattern is used heavily in recommendation systems and leaderboard generation.

02 Locating Elements — where() and nonzero()

```
sales = np.array([5000, 0, 12000, 0, 8500, 0, 3200])
```

Daily sales — zeros on holidays

```
np.where(sales == 0)          -> (array([1, 3, 5]),) — indices of zero-sales days
np.nonzero(sales)            -> indices of all non-zero elements -> [0,2,4,6]
np.where(sales > 5000, 'Good Day', 'Slow Day')
```

Label each day based on performance

```
np.any(sales == 0)          -> True — are there any zero-sales days?
np.all(sales > 0)           -> False — not every day had sales
np.all(sales >= 0)          -> True — no negative values (valid data check)
```

TRY THIS LIVE

Create an array of 15 random exam scores (30–100). Use `np.where()` to find indices of failing scores (<50). Use `np.any()` to check if anyone scored 100. Use `np.all()` to check if everyone passed (>=50).

03 Handling NaN and Infinite Values

Real-world data loaded into NumPy arrays often contains NaN (Not a Number) values — representing missing data — and sometimes infinite values from division by zero or log of zero. NumPy treats NaN as 'toxic': any arithmetic involving NaN returns NaN. You must detect and handle these before computing statistics.

```
data = np.array([10.0, np.nan, 30.0, np.inf, 50.0, np.nan])
```

Array with NaN and Inf

```
np.isnan(data)              Boolean mask -> [F, T, F, F, F, T]
np.isinf(data)              Boolean mask -> [F, F, F, T, F, F]
np.isfinite(data)           True only for normal finite numbers
data[np.isnan(data)] = 0    Replace NaN with 0 (simple imputation)
np.nanmean(data)            Mean ignoring NaN values
np.nanstd(data)             Std ignoring NaN
np.nansum(data)             Sum ignoring NaN
np.nanmedian(data)          Median ignoring NaN
```

04 Clipping Outliers

`np.clip()` caps all values in an array between a minimum and maximum — any value below the floor is set to the floor, any value above the ceiling is set to the ceiling. This is one of the fastest outlier-capping techniques in data preprocessing.

```
ages = np.array([25, 130, 31, 0, 45, 999, 28])
```

Age column with obvious entry errors

```
np.clip(ages, 18, 80)
```

Cap between 18 and 80 -> [25, 80, 31, 18, 45, 80, 28]

```
revenue = np.array([500, 50000, 1200, 999999, 3400])
```

Revenue with extreme outlier

```
np.clip(revenue, 0, np.percentile(revenue, 95))
```

Cap at 95th percentile — Winsorisation technique

PRO TIP — Winsorisation with clip()

Capping values at the 95th (or 99th) percentile instead of a fixed number is called Winsorisation. It removes extreme outliers without deleting rows. Combining `np.percentile()` and `np.clip()` achieves this in two lines — and it's a standard technique in financial and e-commerce analytics.

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Data cleaning with arrays (IIT Madras)

Link — <https://nptel.ac.in/courses/106106145>

WHERE TO GO NEXT

You've now covered the full NumPy toolkit. The final session brings everything together in an end-to-end mini case study.

Explore deeper — NumPy sorting and searching docs:

<https://numpy.org/doc/stable/reference/routines.sort.html>

Try on your own — Generate a 20-element random float array with a few manually inserted `np.nan` values. Compute the `nanmean` and `nanstd`, then replace `NaN` with the `nanmean`, then clip the result between the 5th and 95th percentile.

Connect the dots — Session 09 is the NumPy mini case study — you will use everything from Sessions 01–08 on a real structured dataset.

Going further — Study `np.partition()` and `np.argpartition()` — efficient alternatives to full sorting when you only need the top-k or bottom-k values.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a data cleaning expert. In simple Hinglish, explain how to detect and handle NaN values in a NumPy array using an example of missing sales data for a Gujarat retail chain. Keep it under 150 words.

SESSION

09

90 MIN · APPLIED PROJECT

Mini NumPy Case Study

◆ Learning Objectives

- Load a structured dataset into a NumPy array and inspect its attributes
- Apply data cleaning — handle NaN values, clip outliers, and fix data types
- Perform vectorised calculations for revenue, margin, and growth
- Use indexing and boolean filtering to answer business questions
- Generate a summary report using aggregate and statistical functions

01 Case Study Overview

This session is a guided end-to-end project. You will work with a small structured product-sales dataset — similar in structure to what you will encounter in your first analytics job. Every step uses NumPy exclusively. No Pandas, no Excel. This reinforces that NumPy alone is sufficient for structured numerical analysis when the dataset fits in memory.

CONCEPT — The Dataset

The dataset simulates 10 SKUs (products) sold across 4 cities over 3 months. Columns: Product ID, Base Price (Rs), Units Sold (Month 1-3), Cost Price (Rs). We will compute: Monthly Revenue, Total Revenue, Profit Margin %, Month-on-Month Growth %, and identify the top-performing product and city.

02 Step 1 — Build the Dataset

```
import numpy as np
np.random.seed(2024)
n_products = 10
base_price = np.random.randint(200, 3000, size=n_products).astype(float)
cost_price = base_price * np.random.uniform(0.4, 0.7, size=n_products)
units_m1 = np.random.randint(50, 500, size=n_products).astype(float)
units_m2 = np.random.randint(50, 500, size=n_products).astype(float)
units_m3 = np.random.randint(50, 500, size=n_products).astype(float)
units_m1[[2, 7]] = np.nan
```

Standard import

Reproducible randomness

Number of SKUs

Random prices Rs 200–Rs 3000

Cost = 40–70% of selling price

Units sold — Month 1

Units sold — Month 2

Units sold — Month 3

Inject 2 missing values for realism

```
print('Base prices:', base_price)
```

Inspect the dataset

03 Step 2 — Clean the Data

```
print('NaN in Month 1:', np.isnan(units_m1).sum())
```

Count missing values

```
median_m1 = np.nanmedian(units_m1)
```

Compute median excluding NaN

```
units_m1 = np.where(np.isnan(units_m1), median_m1, units_m1)
```

Impute NaN with median

```
print('NaN after imputation:', np.isnan(units_m1).sum())
```

Verify — should be 0

```
units_m1 = np.clip(units_m1, 0, np.percentile(units_m1, 95))
```

Clip extreme outliers at 95th percentile

04 Step 3 — Compute Revenue & Profit

```
rev_m1 = base_price * units_m1
```

Revenue — Month 1 (element-wise multiply)

```
rev_m2 = base_price * units_m2
```

Revenue — Month 2

```
rev_m3 = base_price * units_m3
```

Revenue — Month 3

```
total_revenue = rev_m1 + rev_m2 + rev_m3
```

Total 3-month revenue per product

```
total_units = units_m1 + units_m2 + units_m3
```

Total units sold per product

```
total_cost = cost_price * total_units
```

Total cost per product

```
profit = total_revenue - total_cost
```

Gross profit per product

```
margin_pct = (profit / total_revenue) * 100
```

Margin % per product

```
print('Profit Margins %:', np.round(margin_pct, 2))
```

Print rounded margins

05 Step 4 — Growth Analysis & Business Answers

```
growth_m2 = ((rev_m2 - rev_m1) / rev_m1) * 100
```

MoM growth Month 1->2 (%)

```
growth_m3 = ((rev_m3 - rev_m2) / rev_m2) * 100
```

MoM growth Month 2->3 (%)

```
print('Top product by revenue:', np.argmax(total_revenue))
    Index of best-performing SKU
print('Highest margin product:', np.argmax(margin_pct))
    Index of most profitable SKU
print('Products with >40% margin:', np.where(margin_pct > 40))
    Which SKUs are high-margin?
print('Avg monthly revenue:', np.round(np.mean([rev_m1, rev_m2, rev_m3], axis=0), 2))
    Average revenue per product across 3 months
print('Best growth M2->M3:', np.argmax(growth_m3))
    Which product grew most in Month 3?
```

06 Step 5 — Print the Summary Report

```
print('='*55)                                Divider line
print('  PRODUCT PERFORMANCE SUMMARY — Q1 2024')
    Report header
print('='*55)                                Divider line
print(f'Total portfolio revenue: Rs {total_revenue.sum():.0f}')
    Grand total revenue
print(f'Average margin: {margin_pct.mean():.1f}%')
    Portfolio average margin
print(f'Highest revenue product: SKU {np.argmax(total_revenue)+1} — Rs
{total_revenue.max():.0f}')
    Best SKU
print(f'Best margin product:      SKU {np.argmax(margin_pct)+1} —
{margin_pct.max():.1f}%')
    Most profitable SKU
print(f'Fastest growing in M3:    SKU {np.argmax(growth_m3)+1} — {growth_m3.max():.1f}%
growth')
    Best growth product
```

PRO TIP — NumPy as a Foundation, Not Just a Tool

Everything you just did — cleaning, transforming, computing, filtering, ranking — is exactly what Pandas does under the hood, on top of NumPy arrays. When you face performance issues in Pandas or need to understand why something works, your NumPy knowledge is what saves you. This case study is both an exercise and a preview of the Pandas module ahead.

CASE STUDY — D-Mart Supply Chain Analytics

Context — D-Mart (Avenue Supermarts), headquartered in Powai, Mumbai, operates 300+ stores across

India including multiple outlets in Gujarat (Ahmedabad, Vadodara, Surat, Rajkot).

Challenge — The supply chain analytics team needs to monitor which SKUs have declining sales velocity month-on-month and which are trending up, to optimise reorder quantities.

What they did — Using NumPy arrays of units-sold per SKU per month, they computed MoM growth rates using vectorised division, flagged declining SKUs (growth < -10%) using boolean indexing, and computed a rolling 3-month average using `np.mean()` with `axis`.

Result — The team was able to automate SKU-level reorder alerts, reducing stockout incidents by 18% and overstock situations by 22% in a 6-month pilot across Ahmedabad stores.

Takeaway — Vectorised NumPy operations on sales arrays — growth rates, moving averages, threshold flags — are the engine behind retail inventory analytics.

Source: D-Mart Annual Report 2022-23; analytics methodology reconstructed for educational purposes

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming, Data Structures and Algorithms using Python — Full course (IIT Madras, Prof. Madhavan Mukund)

Link — <https://nptel.ac.in/courses/106106145>

“Every Pandas DataFrame is a NumPy array with column names. Master NumPy first — and Pandas becomes obvious.”

WHERE TO GO NEXT

You have completed the NumPy module. Carry forward the habit of vectorised thinking — no unnecessary loops, operations on whole arrays at a time.

Explore deeper — NumPy for absolute beginners:
https://numpy.org/doc/stable/user/absolute_beginners.html

Try on your own — Extend this case study: add a discount column (random 0–20%) and recompute actual revenue and margins after discount. Which product becomes unprofitable below 15% discount?

Connect the dots — Module 8 — Pandas — builds directly on this foundation. Every `loc`, `groupby`, and `apply` you write in Pandas runs on NumPy arrays underneath.

Going further — Explore NumPy's structured arrays (`np.dtype` with named fields) for working with mixed-type tabular data without Pandas.

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a data analytics mentor. In simple Hinglish, explain how a TOPS student would explain this NumPy case study in a job interview — focusing on what business problem was solved, which NumPy functions were used, and what insight was generated. Keep it under 200 words.

REVIEW



INTERVIEW PREP · MODULE 7

Interview Questions

◆ Working with NumPy

Test your understanding of Working with NumPy. Rehearse 60-second answers. Compare your responses against the model answers.

1

What is NumPy and why is it popular among data analysts?

Think about performance, large datasets, and mathematical operations.

MODEL ANSWER

NumPy (Numerical Python) is for numerical computing. Arrays are faster, more memory-efficient than Python lists for large datasets. Vectorization: operations on entire arrays without loops.

2

Explain the main difference between a Python list and a NumPy array.

Consider how data is stored and processed in each.

MODEL ANSWER

`np.array([1,2,3])` creates from list. `np.arange(0,10,2)` creates sequence. `np.linspace(0,1,5)` creates 5 evenly-spaced values. `np.zeros/ones/empty` create filled arrays.

3

What is vectorization in NumPy, and why is it important for data analysis?

How does it affect the speed of operations on large datasets?

MODEL ANSWER

ndim dimensions (1D=vector, 2D=matrix, 3D=tensor), shape (size per dimension), size (total elements), dtype (data type), itemsize (bytes per element).

4

How would you compare the performance of a Python list and a NumPy array when performing element-wise addition on large datasets?

MODEL ANSWER

Arithmetic: `+`, `-`, `*`, `/`, `//`, `**`, `%` element-wise. Comparison: `==`, `!=`, `>`, `<` return boolean arrays. Logical: `np.logical_and`, `np.logical_or`, `np.logical_not`.

5

How do you create a NumPy array from a regular Python list?

MODEL ANSWER

`np.add/subtract/multiply/divide`, `np.power`, `np.mod`, `np.sqrt`, `np.exp`, `np.log`. Trig: `sin`, `cos`, `tan`. Rounding: `round`, `floor`, `ceil`.

6

Write code to create a NumPy array containing numbers from 0 to 9 (inclusive).

MODEL ANSWER

Basic: `arr[0]` first element, `arr[0,1]` row 0 col 1. Slicing: `arr[1:3]` rows 1-2, `arr[:, 0]` all rows col 0. Boolean: `arr[arr > 5]`.

7

What is the difference between `np.arange()` and `np.linspace()` when creating arrays?

Think about how the spacing between elements is defined.

MODEL ANSWER

`np.concatenate`, `vstack` (stack rows), `hstack` (stack columns). `np.split`, `array_split` divide arrays. `np.insert` adds elements, `np.delete` removes.

8

How would you create an array of 5 equally spaced values between 1 and 10 using NumPy?

MODEL ANSWER

`np.unique` returns sorted unique values. `np.sort` sorts, `np.argsort` returns sort indices. `np.where` finds matching indices. `np.isnan`, `isinf` check special values.

9

Which function would you use to create an array filled with zeros? What about an array filled with ones?

MODEL ANSWER

NaN (not a number) for missing data. `np.nan`, `np.isnan()` detect. `np.nanmean`, `nanstd`, `nansum` compute ignoring NaN. `np.clip` limits values to range.

10

Write code to generate a 3x3 identity matrix using NumPy.

MODEL ANSWER

Vectorization: `arr * 2` multiplies all elements vs loop multiply each. Vectorized 100× faster. Use NumPy functions instead of loops. Critical for performance.

11

How would you generate a 1D array of 10 random integers between 1 and 100 using NumPy?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

12

What is the purpose of `np.random.seed()`? When should you use it?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

13

How can you check the number of dimensions and the shape of a NumPy array?*Look for attributes that describe the array's structure.***MODEL ANSWER**

See lecture materials for detailed answer on this topic.

14

If you have a NumPy array of shape (4, 3), how would you reshape it to (2, 6)?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

15

What is the difference between `ravel()`, `flatten()`, and `resize()` in NumPy?*Consider whether the result is a copy or a view, and if the original array is affected.***MODEL ANSWER**

See lecture materials for detailed answer on this topic.

16

How do you transpose a 2D NumPy array?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

17

Show how to perform element-wise addition and multiplication between two NumPy arrays of the same shape.**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

18

What is broadcasting in NumPy? Give a simple example where it is useful.*Think about operations between arrays of different shapes.***MODEL ANSWER**

See lecture materials for detailed answer on this topic.

19

Explain the difference between element-wise operations and aggregate operations in NumPy.**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

20

How would you use NumPy to check which elements of an array are greater than 10?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

21

Write code to calculate the square root of every element in a NumPy array.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

22

How do you find the mean and standard deviation of a NumPy array?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

23

What is the difference between `np.sum()` and `np.cumsum()`?

Think about the type of result each function returns.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

24

How would you round all values in a NumPy array to two decimal places?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

25

Write code to find the maximum value and its index in a NumPy array.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

26

How do you access the third element of a 1D NumPy array?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

27

Given a 2D array, how do you select all elements from the second row?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

28

Explain how negative indexing works in NumPy arrays.

Focus on what negative indices refer to.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

29

Write code to select every second element from a NumPy array using slicing.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

30

What is fancy indexing in NumPy? Provide a simple example.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

31

How would you use boolean indexing to select all even numbers from a NumPy array?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

32

Describe how you would use `np.where()` to find the indices of all elements in an array that are less than 5.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

33

How do you concatenate two 1D arrays horizontally using NumPy?

Which function joins arrays along the existing axis?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

34

What is the difference between stacking and concatenation in NumPy?

Consider how the dimensions of the result may change.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

35

Write code to split a 1D NumPy array into three equal parts.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

36

How can you insert a new value at a specific position in a NumPy array?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

37

Explain the difference between a copy and a view in NumPy. Why does it matter?*Think about what happens when you modify the new array.***MODEL ANSWER**

See lecture materials for detailed answer on this topic.

38

How would you find all unique values in a NumPy array?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

39

What does `np.argsort()` return and how can it be useful?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

40

Write code to find the indices of all non-zero elements in a NumPy array.**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

41

How do you check if any or all elements of an array meet a certain condition using NumPy?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

42

What is the purpose of `np.isnan()`, `np.isinf()`, and `np.isfinite()`?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

43

How would you replace all NaN values in an array with the mean of the non-NaN values?**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

44

Describe a step-by-step approach to cleaning and analyzing a small dataset using NumPy arrays (for example, prices and quantities).**MODEL ANSWER**

See lecture materials for detailed answer on this topic.

45

If you have arrays of prices and quantities, how would you calculate the total value for each product using NumPy?

MODEL ANSWER

See lecture materials for detailed answer on this topic.

46

How could you use an AI tool like ChatGPT or GitHub Copilot to help you write or debug NumPy code? What would you do to make sure the code suggestions are safe and correct?

Think about verifying results, understanding code, and not copying blindly.

MODEL ANSWER

See lecture materials for detailed answer on this topic.

47

Imagine you have a large dataset with missing values. How might AI-powered tools help you handle and clean this data using NumPy, and what would you check before trusting their output?

MODEL ANSWER

See lecture materials for detailed answer on this topic.